Hoisting in JavaScript

Hoisting in JavaScript is a behavior where **variable and function declarations are moved to the top of their scope** (either global or function scope) **before code execution**.

However, **only declarations are hoisted, not initializations**.

🔹 Variable Hoisting

`var`

```
javascript

console.log(a); // undefined
var a = 5;
```

JavaScript internally does:

```
javascript

var a;       // Declaration is hoisted
console.log(a); // undefined
a = 5;       // Initialization stays in place
```

`let` and `const`

```
javascript

console.log(b); // ReferenceError: Cannot access 'b' before initialization
let b = 10;
```

Even though `let` and `const` are hoisted, **they are in a "temporal dead zone" (TDZ)** from the start of the block until the declaration is encountered. Accessing them before declaration throws an error.

🔹 Function Hoisting

Function Declaration (hoisted)

```
javascript

sayHello(); // "Hello!"
function sayHello() {
  console.log("Hello!");
}
```

Function Expression (not hoisted)

```javascript
sayHi(); // TypeError: sayHi is not a function
var sayHi = function () {
  console.log("Hi!");
};
```

Only **function declarations** are hoisted, not **function expressions**.

✅ Summary

Type	Hoisted?	Usable before declaration?	Notes
`var`	✅ Yes	⚠️ Yes (but undefined)	Initialized to `undefined`
`let` / `const`	✅ Yes (TDZ)	❌ No (ReferenceError)	In Temporal Dead Zone until declared
Function Decl.	✅ Yes	✅ Yes	Fully hoisted
Function Expr.	❌ No	❌ No	Treated like variables

Let me know if you want visual diagrams or animations to understand it better.

Great question!

Hoisting is *real* — it's not just a theoretical concept. It's **how the JavaScript engine actually works under the hood** during the **compilation phase** *before* your code runs.

✅ Here's what actually happens:

When JavaScript runs your code, it goes through two main phases:

1. **Compilation Phase**
   - It scans through your code.
   - All **variable declarations** (`var`, `let`, `const`) and **function declarations** are *registered* in memory.
   - But:
     - `var` is initialized as `undefined`.
     - `let` and `const` are placed in a **Temporal Dead Zone (TDZ)** — they exist but can't be accessed yet.
     - Function declarations are fully hoisted with their code.
2. **Execution Phase**
   - The code runs line by line.
   - Assignments and logic execute now.

🔍 Real Example (Behind the scenes):

Your Code:

```javascript

console.log(x); // ?
var x = 10;
```

JavaScript Internally:

```javascript

var x;           // Hoisted
console.log(x);  // undefined
x = 10;          // Initialization happens here
```

So when you say:

```javascript

console.log(x);
```

JavaScript **already knows `x` exists**, but it's set to **`undefined`**.

🔥 Proof: Try This in Console

Paste this into your browser console:

```javascript

console.log(myVar);
var myVar = 42;
```

You'll see:

```javascript

undefined
```

Now try this:

```javascript

console.log(myLet);
let myLet = 42;
```

You'll get:

```pgsql

ReferenceError: Cannot access 'myLet' before initialization
```

✅ Conclusion:

- 🔹 Hoisting is **not just a concept** — it's a **real part of how JavaScript works** at the engine level during compilation.

Let me know if you want to see a **step-by-step visual flow** of what JavaScript does behind the scenes.

It looks like you've uploaded a ZIP file named `Day17.zip`. To help you effectively, I'll first extract and review the contents of the ZIP file. Then I'll prepare the **notes** based on what's inside.

Let me proceed to extract the ZIP.

The ZIP file contains the following relevant files in the `Day17` folder:

1. `first.js`
2. `second.js`
3. `index.js`
4. A PDF: `Untitled Notebook (188).pdf`

Would you like me to prepare notes from:

- Just the **JavaScript files**
- Just the **PDF**
- Or **both** combined?

Let me know your preference, and I'll proceed accordingly.

The PDF appears to contain **poorly extracted text** (likely from handwritten notes or a scanned document), which makes it hard to parse meaningfully through text alone. But don't worry — I'll still extract relevant points from the JavaScript files and reconstruct the main ideas (like hoisting, execution context, TDZ, etc.) based on patterns and file names.

Now, I'll parse the contents of the JavaScript files: `index.js`, `first.js`, and `second.js`.

Based on the extracted **JavaScript files** and partially readable **PDF**, here are your **compiled notes** on core JavaScript concepts, especially focused on **hoisting**, **execution context**, and **temporal dead zone (TDZ)**.

📝 JavaScript Day 17 — Notes Summary

🔹 1. Hoisting

Hoisting is JavaScript's default behavior of moving declarations (not initializations) to the top of the current scope before code execution.

✅ Example from `index.js`:

```
javascript

console.log(x); // undefined
console.log(y); // ReferenceError: Cannot access 'y' before initialization
```

```
var x = 10;
let y = 20;
```

🔍 Explanation:

- `var` is **hoisted** and initialized to `undefined`.
- `let` is **hoisted** but not initialized (TDZ → Temporal Dead Zone).

🔹 2. Temporal Dead Zone (TDZ)

The TDZ is the time between entering the scope and when the variable declared with `let` or `const` is initialized.

✅ Example from `first.js`:

```
javascript

let z;
var x = undefined;
let y;

console.log(x); // undefined
z = 50;

x = 10;
y = 20;

console.log(x); // 10
```

- `x` is declared with `var`, so it is hoisted and initialized with `undefined`.
- `z` and `y` are declared with `let`, so they are hoisted but remain in TDZ until initialization.

🔹 3. Function Hoisting

✅ Function Declaration (hoisted):

```
javascript

// greet(); // Uncommenting will work

// function greet() {
//     console.log("Hello World");
// }
```

- Function declarations are hoisted **with their entire definition**.

❌ Function Expression (not hoisted):

```
javascript

var meet = function () {
    console.log("Hello Meet");
};
```

```
meet(); // Works because it is called after declaration
```

- Function expressions are treated like variables.
- `meet` is hoisted as `undefined`, so calling it before its definition throws an error.

🔹 4. Memory Allocation Phase (Conceptual)

During the creation phase of the execution context:

- `var` variables are assigned `undefined`.
- `let` and `const` are placed in TDZ.
- Function declarations are stored with full code.

🔹 5. Execution Context

Each time a function is invoked or a script runs, an execution context is created. It has:

- **Memory/Variable Environment** (for hoisting)
- **Code Execution Phase**

✅ Summary Table

Keyword/Type	Hoisted	Initialized	Accessible Before Declaration
`var`	Yes	`undefined`	Yes
`let` / `const`	Yes	No (TDZ)	No (throws ReferenceError)
Function Declaration	Yes	Yes	Yes
Function Expression	Yes	`undefined`	No

Would you like this compiled into a downloadable PDF or DOCX file for study?